2-11-2026

MDRS

Lab2

Victor Manuel Sandoval Maciel
Adolfo Meza Ibarra
Diseño, verificación y validación de sistemas digitales

Content

Entregables. 3
Desarrollo. 4
Diagrama de MDRS_UART. 4
Descripción de módulos. 5
MDRS. 5
UART(General). 7
UART Rx. 9
UART_Tx. 10
General 10
Algoritmos implementados 11
Multiplicación / Square - Booth. 11
División - Restoring. 12
Raíz - Seudocódigo 13
FSM de MDRS_UART. 15
FSM de UART_Rx_Tx. 15
Resultados de Síntesis. 16
Timing con freq máxima 16
Tamaño dathapath 16
Síntesis de datapath y RTL. 17
RTL 17
SignalTab. 17
MDRS Interfaz y Testbench con interfaz 23
Interfaz mdrs_if 23
Clase mdrs_tester 23
Testbench mdrs_class_tb 23
Simulación por operación 24
Modelo de referencia (Extra 10%). 26
Implementación. 27
Mapeo de pines. 27
Capturas de funcionamiento de MDRS. 27
Multiplicación 27
División 28
Raíz 28

Square 28
Validación. 29
Python Script auto-check (Extra 10%). 31
Datapath estructural. 31
Referencias. 34
Anexo. Prompts de IA utilizados. 34
Prompt 1. 34
Prompt 2. 35
Prompt 3. 37
Prompt 4. 39
Prompt 5. 41

Entregables.

Deliverables (at team's github repository):

- A pdf report including:

- Diagrams and explanations of MDRS architecture proposed. Include a functional description of each module. If an FSM is used, include its diagram.
- Simulation screenshots explaining each signal and how MDRS works.
- Synthesis results including Logic utilization and maximum frequency allow for your design.
- Synthesis of the data path where HW limitations are shown in either reports or RTL view.
- FPGA implementation screenshots/log files results.
    - Extra points (10%) Modify python script to automatically check 100% of operations and values.
    - Extra points (20%) Change mdrs_driver_10bits.sv with your own implementation that follows a different strategy.
    - Extra points (20%) Implement the data path in a structural way where units are not larger than:
        - Gates
        - Multiplexors
        - Registers
        - Operations (add/sub/shift/mask)
    - SignalTab screenshot with central register value on it.

- All source files with enough comments.

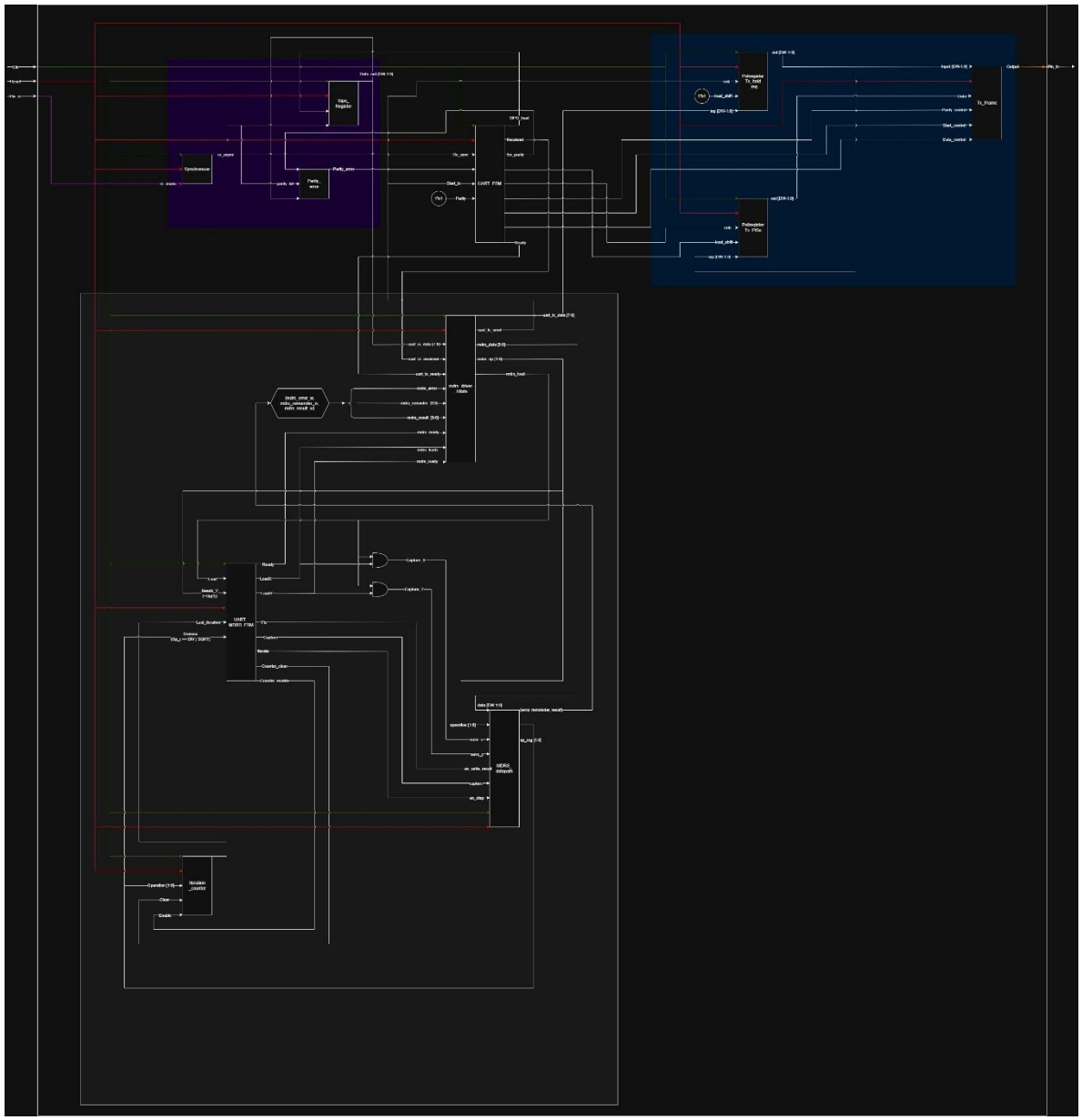
- At least one package must be used for the implementation of the product.

- Testbench file with most important scenarios you considered to test.

- This must use an interface to connect the MDRS block to the testbench.
- It must use at least one class and interacts with MDRS through high-level methods calls in the testbench.
- Extra points (10%) Implement a reference model comparison methodology.
- Extra points (20%) Implement a SVA methodology (verifying all timing relations among load, loadX, loadY, ready and error signals and -1 values in output for error signaling).
- Extra points (30%) Implement a UVM methodology.
- Extra points are considered with the maximum possible value (if UVM + SVA are implemented it takes highest score which is 30%).

Desarrollo.

Diagrama de MDRS_UART.



Descripción de módulos.

MDRS.

- DMRS_Datapath.

Descripción.

Datapath del MDRS para multiplicación, división, raíz y square.
Las cuatro operaciones comparten los registros (X, Y, acumulador, result, remainder, error y operación) y un sumador/restador de 2*DW+1 bits, y sus entradas se eligen con un mux según la operación.
Operación = 0 y 3.
Multiplicación y square: Booth, una iteración por ciclo. X es el multiplicando, Y (o X en square) el multiplicador. El resultado se captura con capture.
Operación = 1.
División restoring se ejecuta sobre valores absolutos. En el ciclo de X se obtiene |X|. Los signos del cociente y del residuo se aplican en el ciclo "en_write_result".
Operación = 2.
La raíz se ejecuta digito por digito, 2 bits de X por iteración (y_reg hace de registro de desplazamiento). En "en_write_result" se corrige el residuo.

Error deja result y remainder en -1 (resultado no representable, división entre 0 o raíz de un negativo).

Parámetros.

DW: Ancho de los datos.

Entradas.

clk:	Reloj del sistema (posedge).
rst:	Reset del sistema (asíncrono y activo en 0).
save_x:	Captura X y la operación (ciclo de LoadX con load).
save_y:	Captura Y (ciclo de LoadY con load).
en_step:	Ejecuta una iteración del algoritmo.
en_write_result:	Escritura final de div y sqrt.
capture:	Captura el resultado de mult y square.
operation [1:0]:	Selecciona la operación (0 = mult, 1 = div, 2 = sqrt, 3 = square).
data [DW-1:0]:	Operando presente.

Salidas.

op_reg [1:0]:	Operación registrada.
result [DW-1:0]:	Resultado de la última operación.
remainder [DW-1:0]:	Residuo de la última operación.
error:	La operación no se pudo calcular (result y remainder quedan en -1).

- MDRS_wrapper.

Descripción.

Envuelve UART_MDRS_FSM, Iteration_counter y MDRS_datapath_estructural con los puertos de la práctica. Son las mismas conexiones que hay en UART_DMRS (Needs_Y = ~operation[1], Division = op_reg es div o sqrt, save_x = loadX & load, save_y = loadY & load), así que lo que se simula aquí es exactamente lo que va en la FPGA pero sin UART. Es el DUT del testbench con interfaz.

Parámetros.

DW: Ancho de los datos.

Entradas.

clk, rst:	Reloj (posedge) y reset asíncrono activo en 0.
load:	Pulso de carga de operando.
operation [1:0]:	0 = mult, 1 = div, 2 = sqrt, 3 = square.
data [DW-1:0]:	Operando presente.

Salidas.

ready, loadX, loadY:	Estado del protocolo (vienen de la FSM).
result, remainder [DW-1:0], error:	Salidas del datapath.

- Sumador_Restador.

Descripción.

Sumador/restador combinacional de DW bits.

Parámetros.

DW: Ancho de los operandos y del resultado.

Entradas.

a [DW-1:0] (signado): Operando A.
b [DW-1:0] (signado): Operando B.
sel: 0 = a + b, 1 = a - b.

Salidas.

- Result [DW-1:0] (signado): Resultado de la operación.

- Iteration-Counter.

Descripción.

Contador de iteraciones del MDRS. Da la señal de Last_iteration para cuando el conteo llega a la última iteración de la operación en curso.

Entradas.

Clk: Reloj del sistema (actúa de posedge).
Reset: Reset del sistema (activo en 0).
Clear: Limpia el conteo (síncrono, activo en alto, tiene prioridad sobre Enable).
Enable: Incrementa el conteo (activo en alto).
Operation: 0 = mult, 1 = div, 2 = sqrt, 3 = square.

Salidas.

Last_iteration: Ultima iteración de la operación.

- UART_MDRS_FSM.

Descripción.

Máquina de estados de MDRS.

Entradas.

Clk:	Reloj del sistema.
Reset:	Reset activo en 0.
Load:	Señal de carga de operando (activa en alto).
Needs_Y:	La operación necesita segundo operando (para mult o div).
Last_iteration:	El contador llego a la última iteración (activo en alto).
División:	La operación en curso termina con un ciclo de corrección (div o sqrt).

Salidas.

Ready:	El sistema está listo para recibir.
LoadX:	Se espera el operando X.
LoadY:	Se espera el operando Y.
Iterate:	Se tiene una iteración del algoritmo.
Fix:	Escritura final con corrección de signos (para div o el residuo sqrt).
Capture:	Captura de result/remainder/error (para mult y square, al terminar).
Counter_clear:	Limpia el contador de iteraciones.
Counter_enable:	Habilita el contador de iteraciones.

UART(General).

- Uart_fsm.

Descripción.

Maquina de estados Moore de UART (para Rx y Tx). Contiene los contadores de baudios y de bits.
En Rx, el reloj de muestreo se alinea con la mitad del bit:
en “RX_START” se espera la mitad del bit de start, y al cambiar de estado el contador se reinicia, de modo que “baud_tick” cae en la mitad de cada bit
Si en IDLE coinciden Start_tx y el inicio de un frame de Rx, tiene prioridad el Tx para no perder el pulso de envió del driver.

Parámetros.

CLKS_PER_BIT: Ciclos de reloj por bit (clk / baudrate).

Diagrama de estados y transiciones.

Entradas.

Clk:	Reloj del sistema (actúa de posedge).
Reset:	Reset (activo en 0).
Rx_sync:	Línea RX sincronizada.
Parity_error:	Señal de error de paridad del frame recibido.
Start_tx:	Inicio de transmisión (activo en bajo 0).
Parity:	Habilita el bit de paridad en el Tx.

Salidas.

SIPO_load:	Carga el bit muestreado al SIPO (en RX_DATA).

En_parity:	Habilita la comparación de paridad (en RX_PARITY).
Received:	Indica que el frame fue recibido sin error.
load_shift:	Habilita carga paralela (en 1) o desplazamiento (en 0) en PISO.
enb:	Enable de actualización en PISO.
Parity_control:	Control hacia Tx Frame para paridad.
Start_control	Señal de inicio hacia Tx Frame.
Data_control:	Control hacia Tx Frame para datos seriales.
Ready:	Bandera de IDLE (el Tx está disponible).

UART Rx.

- Parity_error.

Descripción.

Calcula la paridad de Data_out según PARITY_TYPE (ODD por default, EVEN opcional) y la compara con el bit de paridad recibido. Si en_parity esta inactivo, parity_error se mantiene en 0.

Parámetros:

PARITY_TYPE:	"ODD" (default) o "EVEN".
DW:	Ancho del bus de datos.

Entradas.

Data_out [DW-1:0]:	Datos recibidos para calcular la paridad.
en_parity:	Habilita la comparación de paridad.
parity_bit:	Bit de paridad recibido.

Salidas.

parity_error: 1 cuando la paridad calculada no coincide con parity_bit.

- SIPO_Register.

Descripción.

Registro SiPo que recibe los datos seriales y los almacena, cuando SIPO_load los datos almacenados salen en paralelo.

Entradas.

clk:	Reloj del sistema (actúa de posedge).
rst:	Reset del sistema.
SIPO_load:	Señal de control que permite el paso de datos al fliflop.
rx_sync:	Entrada serial

Salidas.

Data_out [DW-1:0]:	Bus de salida del módulo (en paralelo).

- Synchronizer.

Descripción.

Sincronizador de 3 flip-flops para la entrada RX.

Entradas.

clk:	Clk del sistema.
rst:	Reset del sistema. (Activo en bajo).
rx_async:	Entrada asíncrona que viene directo del transmisor.

Salidas.

rx_sync:	Salida sincronizada (valor del último flip-flop).

UART_Tx.

- Tx_Frame.

Descripción.

Wrap del frame. Coloca los bits de start, datos y stop según las señales de control de la FSM.

Entradas.

Input:	Bus paralelo de datos [DW-1:0] para cálculo de paridad.
Clk:	Señal de reloj.
Reset:	Reset del sistema (activo en bajo).
Data:	Bit de datos serial proveniente del PISO.
Start_control:	Señal que ordena insertar el start bit (Se activa en 0).
Parity_control:	Habilita el uso de Parity bit

Salidas.

Output: salida del módulo a uart tx.

General.

- MDRS_pkg.

Descripción.

Enumeraciones de operación del MDRS.

- Poli_Register_pkg.

Descripción.

Enumeración de comportamientos para PoliRegister.

- Poli_Register.

Descripción.

Registro secuencial “polimórfico”. Permite por medio de un parámetro, cambiar el ancho de palabra (DW) y su modo de operación.

Entradas.

clk:	Reloj del sistema.
sync_reset:	Reset síncrono, activo en alto.
enb:	Enable de captura del registro.
load_shift:	Selector de modo Load/Shift (1 = Carga paralela, 0 = Desplazamiento).
inp:	Bus de datos de entrada paralelo/serie de tamaño [DW-1:0].

Salidas.

out:	Salida del módulo. Puede sacar el bus paralelo [DW-1:0] o un bit.

Algoritmos implementados

Las implementaciones comparten un acumulador y un módulo sumador/restador

Multiplicación / Square - Booth.

Ambas, multiplicación e Square son secuenciales y utilizan el algoritmo de booth, esta toma DW (Data width o ancho de palabra) ciclos, para este caso son 10.

Variables e registros.

-"x_reg" guarda X y funciona como el multiplicando M.

- Hay un "accumulator" y tiene 21 bits (2·DW+1) y se parte así: [20:11] = A, [10:1] = Q (el multiplicador) y [0] = Q_{-1}. Que es el registro "Product" de Booth.

- "adder_unit", calcula A $\pm$ M. Su entrada se elige por "op_now".

Mult y square caen en el default del mux:

"adder_a" = A con signo extendido.

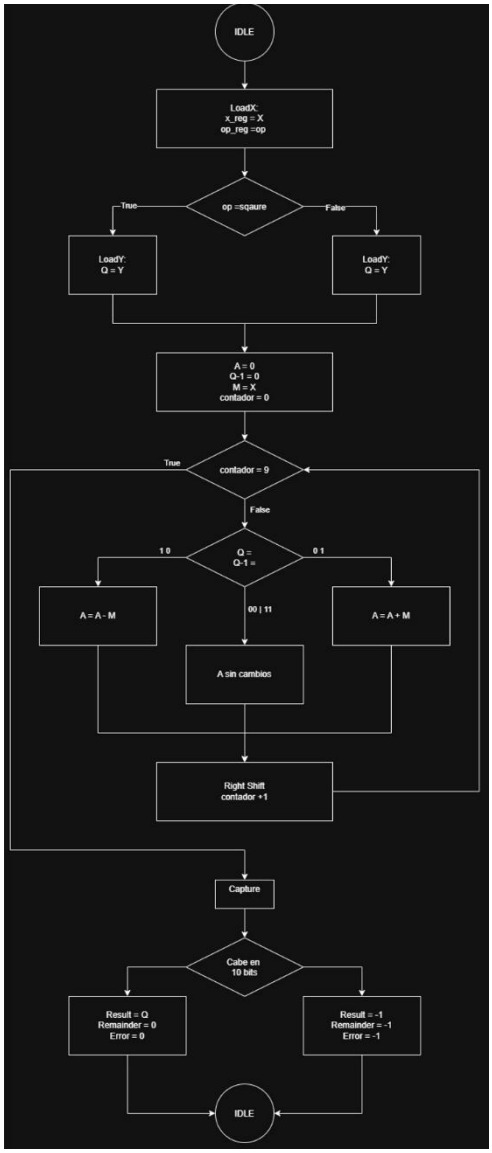
"adder_b" = M con signo extendido.

"adder_sel" = "Subtract_w".

La decisión de Booth sale de "Product" [1:0], o sea {Q_0, Q_{-1}}:

Q y Q_{-1}	El acumulador…
0 1	Se le suma X
1 0	Se resta X
1 1, 0 0	Queda igual

- "Product_next" = { "acc_next" [10:0], "Product" [10:1] } es el desplazamiento aritmético a la derecha. El bit 10 de la suma de 11 bits queda como signo de A.



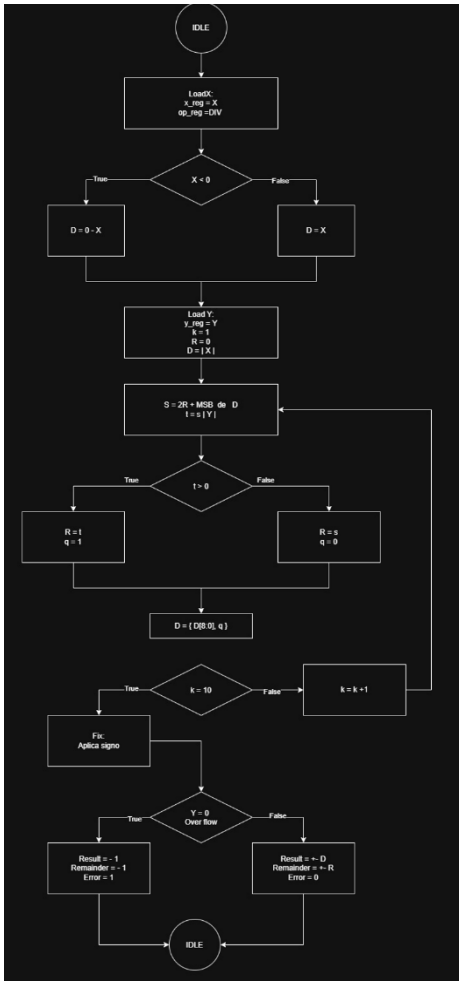
A través de los ciclos podemos describir el cálculo como

Ciclo	Estado	
-	IDLE	Load con X y "save_x"=1. "op_now" toma "operation" porque "op_reg" aún no está escrito. Y "x_reg" = X, "op_reg" = "op" y "accumulator" = {0, X, 0}. Como "Needs_Y" = ~op[1] = 1, la FSM pasa a WAIT_Y.
-	WAIT_Y	Espera Load. Mientras no llegue, no cambia nada. Al llegar Y, "save_y" = 1 hace "Load" = 1 y "accumulator" = {0, Y, 0}. Esto pisa lo que escribió en IDLE y deja X en x_reg como M y Y en Q como multiplicador. La FSM pasa a RUN.

0-8	RUN	Con “en_step” =1, “accumulator” = “Product_d_w” = “Product_next”. Cada ciclo hace una decisión Booth (sumar, restar o nada) y un shift a la derecha. El contador pasa de 0 a 9.
9	RUN	“Iterate” = 0, así que el registro no se actualiza. En este mismo ciclo, de forma combinacional, se hace la iteración 10 y se captura “Product_next” en “result”.
-	IDLE	Ready=1. El driver manda por UART result, remainder y error.

Para los casos en donde la operación no cabe Overflow se calcula sobre “Product_next” [20:10], 11 bits. Si son todos 0 o todos 1, el producto cabe en 10 bits con signo (“Overflow_w” = 0). Con esto se tendría.
result = -1 (todos unos), remainder = -1 y error = 1.
Si no, hay overflow, “result” = “Product_next” [10:1] (el valor de Q), remainder = 0 y error = 0.

División - Restoring.



Registros y sumador en división
“x_reg” guarda X, y también “x_sign”, que se usa en FIX.
“y_reg” guarda Y, el divisor, que se escribe en el ciclo de save_y.
“accumulator” (de 21 bits) se parte: [20:10], en el residuo parcial R (11 bits) y [9:0] es el dividendo restante junto con el cociente.
Cada ciclo se mete un bit del cociente por abajo y sale un bit del dividendo por arriba. El siguiente bit del dividendo siempre es “accumulator”[9].
El sumador calcula “shifted_remainder” – Y en 11 bits, con “shifted_remainder” = {R[9:0], accumulator[9]}.

Si Y > 0, “sel” = ~”y_sign” = 1 y hace shifted − Y.

Si Y < 0, “sel” = 0 y hace shifted + Y, que es lo mismo que restar Y.
“divisor_fits” = ~”adder_sum”[10] indica si la resta no dio negativa.
Convención de signos: la división se hace sobre |X| y |Y| y en FIX se arreglan los signos. El cociente es negativo si X y Y tienen signo distinto y el residuo siempre lleva el signo de X (igual que el % de SystemVerilog y que x - int(x/y)*y en Python), por eso -100 / 7 da -14 y residuo -2.

La actualización es restoring.

Si cabe: “acc” = {sum[10:0], acc[8:0], 1}, o sea el residuo es la resta y el bit del cociente es 1.
Si no cabe: “acc” = {shifted, acc[8:0], 0}, o sea el residuo se restaura y el bit del cociente es 0.

Ciclo	Estado	
-	IDLE	“Load” con X y “save_x” = 1. “op_now” toma “operation”, “x_reg” = X y “op_reg” = “DIV”. El sumador calcula 0 – X, si X es negativo (“sel” = data[9]), y se guarda “acc” = {0, 0, X}. Como “Needs_Y” = 1, la FSM pasa al estado WAIT_Y.
-	WAIT_Y	Espera Load. Al llegar Y, divisor = data y “y_reg” = Y. En este mismo ciclo se hace la iteración 1: “shifted” = {0, |X|[9]}, se resta |Y| y se actualiza el acumulador según “divisor_fits”. La FSM pasa a RUN.
0-8	RUN	Se hace una iteración restoring por ciclo, con divisor = “y_reg”. El contador cuenta de 0 a Last_div = DW−2 = 8. En R8 la FSM ve Last_iteration y Division=1, así que sigue a FIX.
9	FIX	Se aplican signos y se escribe result, remainder y error.
-	IDLE	Ready=1. Los resultados ya están estables en los registros.

Raíz - Seudocódigo

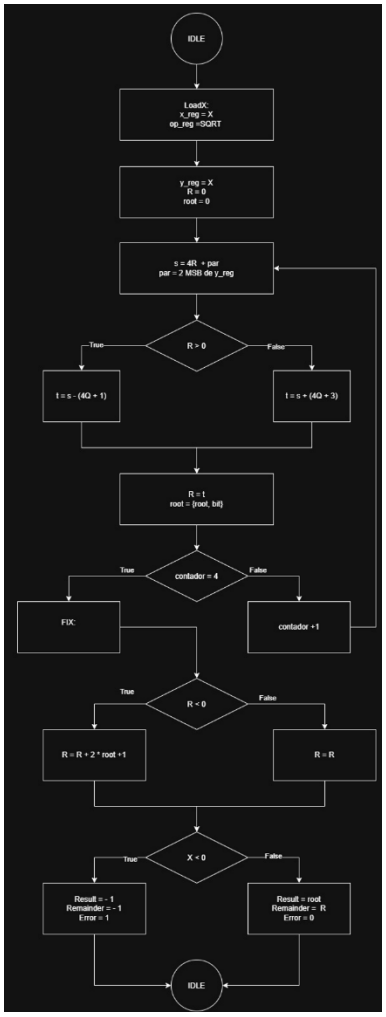
x_reg guarda X, que solo se usa para su signo en FIX.
y_reg guarda una copia de X y funciona como registro de desplazamiento. Cada iteración usa sus 2 bits más altos y luego se recorre 2 posiciones a la izquierda.
accumulator (21 bits) se parte así:
[20:10] es el residuo R con signo y [9:0] es la raíz Q construida hasta ahora. El bit 20 es el signo de R.
Cada iteración usa el sumador único:

a = 4R + par: {acc[18:10], par}.
b = 4Q + 1 si R ≥ 0, o 4 Q + 3 si R < 0: {root[8:0], signo_R, 1}.
sel = ~signo_R: si R ≥ 0 resta, si R < 0 suma.

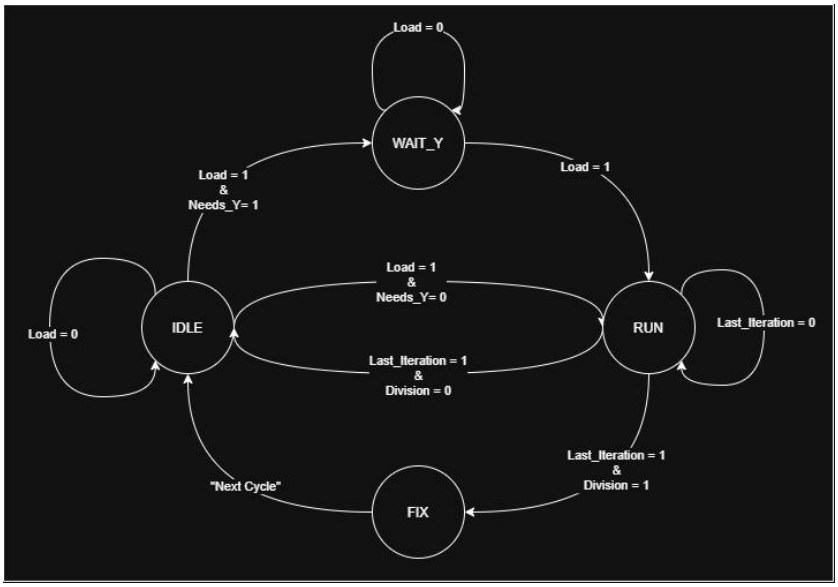
El bit nuevo de la raíz es root_bit = ~adder_sum[10]
acc = { adder_sum[10:0], root[8:0], root_bit }.
R pasa a ser la suma o resta, y la raíz se recorre 1 bit metiendo el bit nuevo por abajo.

Ciclo	Estado	
-	IDLE	Load con X y save_x=1. Se hace x_reg = X, op_reg = SQRT, y_reg = X y acc = 0. Como Needs_Y = ~op[1] = 0, la FSM salta WAIT_Y y va directo a RUN.
0-4	RUN	Una iteración por ciclo, 2 bits de X cada una.

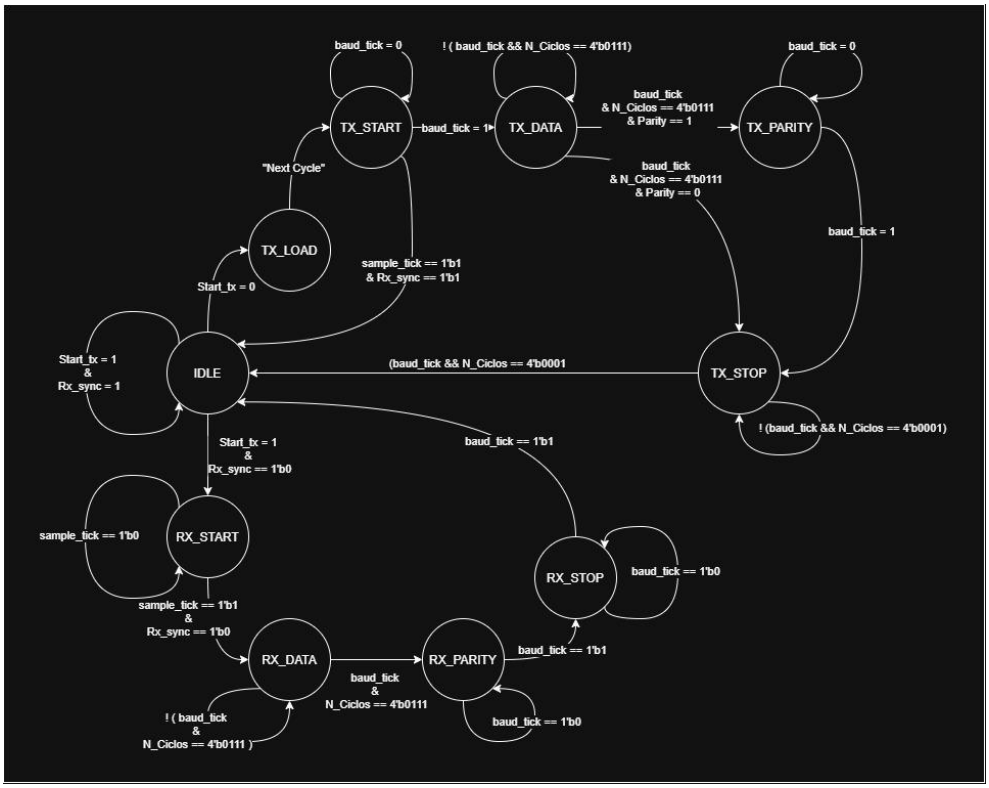
		El contador va de 0 a Last_sqrt = DW/2 − 1 = 4. En el ciclo 4 la FSM ve Last_iteration y Division=1, así que sigue a FIX.
5	FIX	Si R quedó negativo, el sumador hace R + 2·root + 1. Si X era negativo, escribe error.
-	IDLE	Ready=1. result, remainder y error listos para otra operación.



FSM de MDRS_UART.

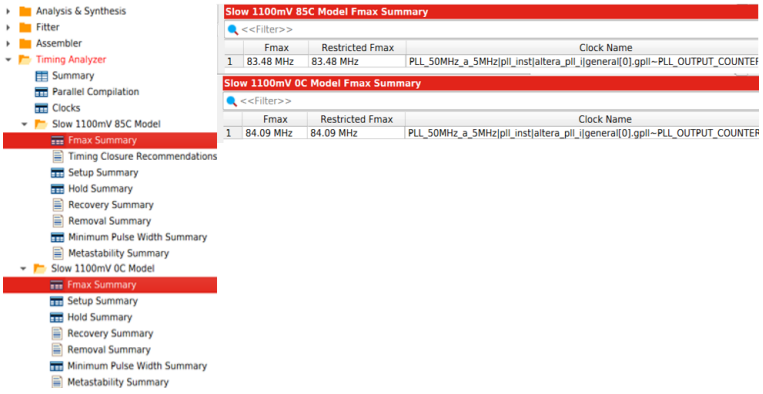


FSM de UART_Rx_Tx.

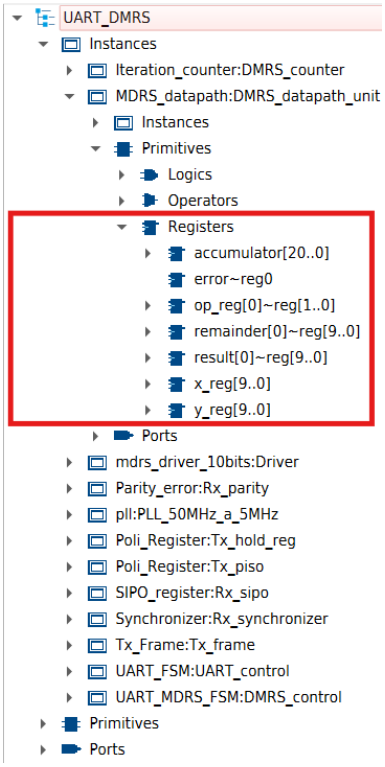


Resultados de Síntesis.

Timing con freq máxima



Tamaño dathapath

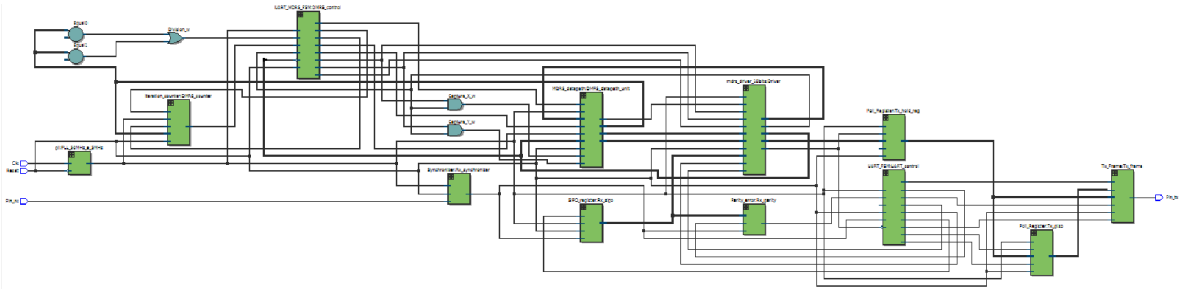


Nombre del registro/s	N. Registros(FlipFlops)
accumulator	21
error	1
op_reg	2
remainder	10
result	10
x_reg	10
y_reg	10
Total	64

Utilizando el tool de RTL viewer y dentro de la instancia de datapath se tiene la catergorias de registros dentro de la instancia. Para esto Dathapath consta de 64 registros

Síntesis de datapath y RTL.

RTL.

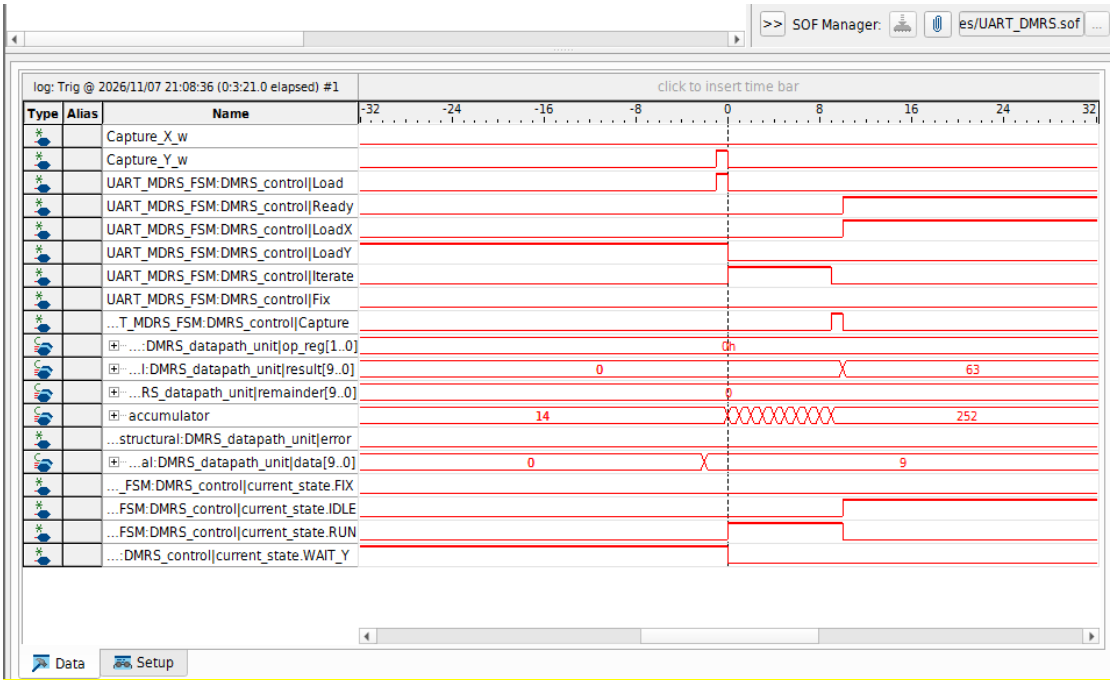


SignalTab.

SignalTap se agregó al proyecto de Quartus (UART_DMRS.stp) para ver el registro central del MDRS trabajando en la FPGA. Se configuró así:
- Reloj de adquisición: la salida del PLL (5 MHz), el mismo reloj del MDRS, para tener una muestra por ciclo.
- 256 muestras, posición del trigger center trigger y flow control Sequential.
- Condición de disparo: current_state.RUN = High. Dispara justo cuando la FSM entra a RUN, o sea cuando llega el último operando.
- Señales: Capture_X_w y Capture_Y_w, las salidas de la FSM (Load, Ready, LoadX, LoadY, Iterate, Fix, Capture), los cuatro estados (IDLE, WAIT_Y, RUN, FIX), op_reg, data, result, remainder, error y el acumulador de 21 bits.

Multiplicación 7 × 9 = 63.

Al entrar a RUN el acumulador trae {0, Y, 0}. En cada ciclo con Iterate se ve la decisión de Booth (suma, resta o nada según los dos bits bajos) y el corrimiento a la derecha; en el ciclo 10 sube Capture y result toma 63 con remainder 0 y error 0. Ready y LoadX regresan a 1 y la FSM a IDLE.




```
>>> m.load(0, 7); m.load(0, 9)
Result = 0 (0x0000)
Remainder =  0 (0x0000)
LoadX = 0
LoadY = 1
Ready = 0
Error = 0
(0, 0, 0, 1, 0, 0)
Result = 63 (0x003f)
Remainder =  0 (0x0000)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(63, 0, 1, 0, 1, 0)
```

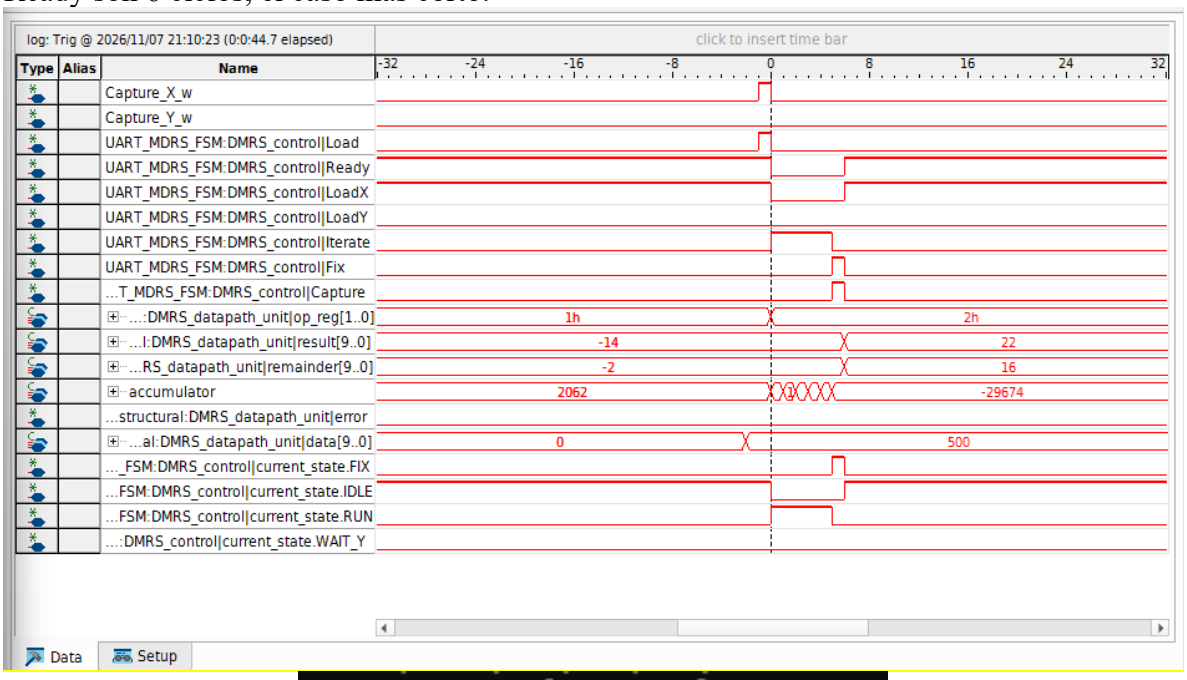
división -100 / 7.

El acumulador parte con |X| = 100 abajo y en cada iteración restoring el residuo parcial se recorre, se le resta 7 y entra el bit del cociente por abajo. Con la última iteración la FSM pasa a FIX un ciclo: ahí se niegan cociente y residuo según los signos y result/remainder cambian a -14 y -2. En la primera captura que se tomó (de la tanda aleatoria) se ve lo mismo con -259 / -144: el acumulador termina en 0x1CC01, que leído por campos es residuo 0x73 = 115 y cociente 0x001 = 1, y después de FIX salen result = 1 y remainder = -115.




```
>>> m.load(1, -100); m.load(1, 7)
Result = 63 (0x003f)
Remainder =  0 (0x0000)
LoadX = 0
LoadY = 1
Ready = 0
Error = 0
(63, 0, 0, 1, 0, 0)
Result = -14 (0x03f2)
Remainder =  -2 (0x03fe)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(-14, -2, 1, 0, 1, 0)
```

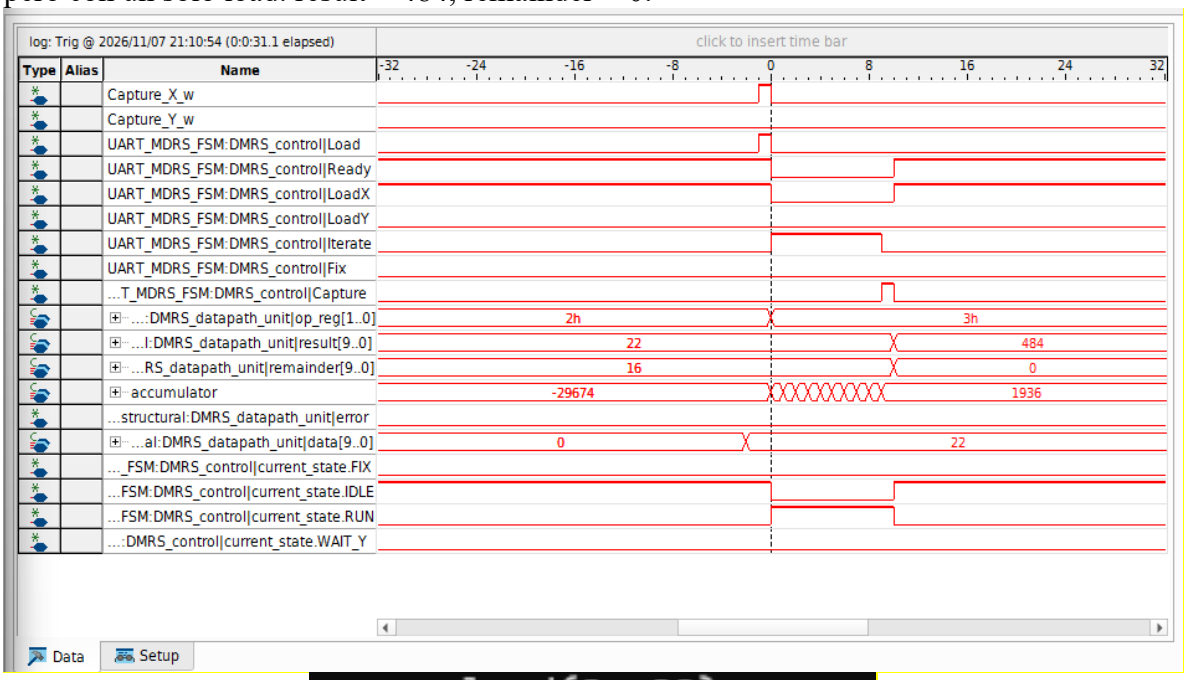
Raíz de 500 = 22, residuo 16.
Sólo hay un load (Capture_X_w) y la FSM va directo a RUN. Son 5 iteraciones, una por cada par de bits de X; se ve el acumulador construyendo la raíz en los bits bajos y el residuo en los altos. En FIX el residuo quedó positivo así que no se corrige. De load a Ready son 6 ciclos, el caso más corto.



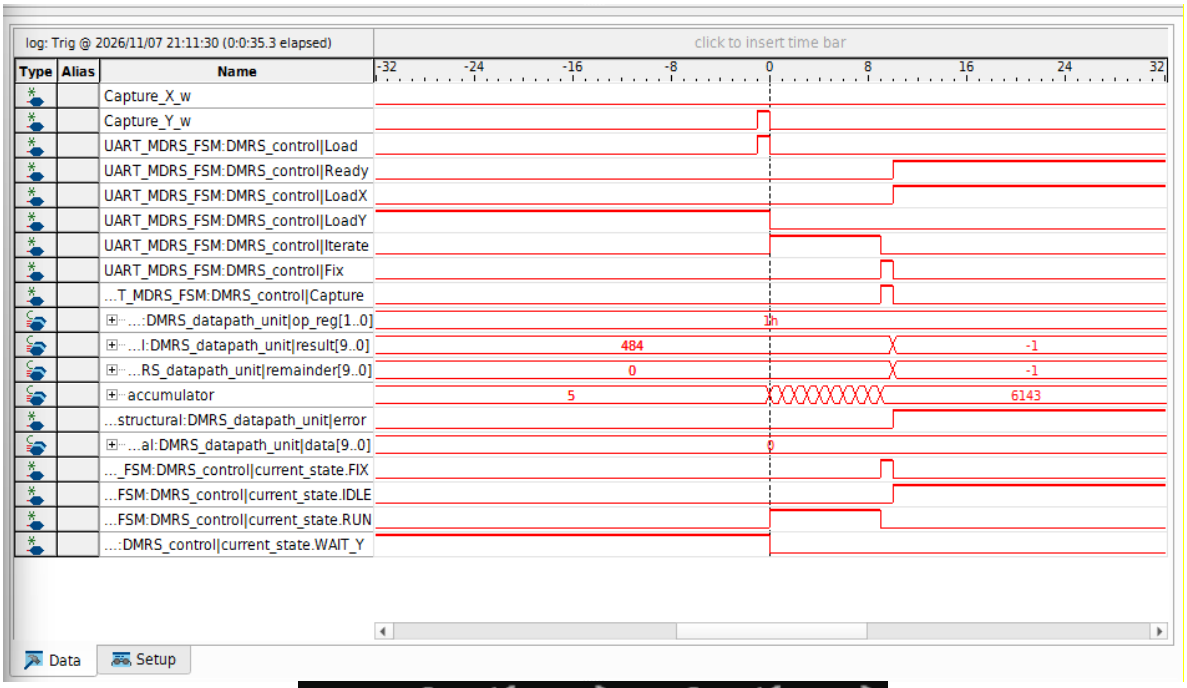

```
>>> m.load(2, 500)
Result = 22 (0x0016)
Remainder =  16 (0x0010)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(22, 16, 1, 0, 1, 0)
```

Cuadrado de 22 = 484.

Mismo camino que la multiplicación (Booth con X como multiplicando y multiplicador), pero con un solo load. result = 484, remainder = 0.




```
>>> m.load(3, 22)
Result = 484 (0x01e4)
Remainder =  0 (0x0000)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(484, 0, 1, 0, 1, 0)
```




```
>>> m.load(1, 5); m.load(1, 0)
Result = 484 (0x01e4)
Remainder =  0 (0x0000)
LoadX = 0
LoadY = 1
Ready = 0
Error = 0
(484, 0, 0, 1, 0, 0)
Result = -1 (0x03ff)
Remainder =  -1 (0x03ff)
LoadX = 1
LoadY = 0
Ready = 1
Error = 1
(-1, -1, 1, 0, 1, 1)
```

MDRS Interfaz y Testbench con interfaz

Para probar el MDRS completo se hicieron cuatro archivos, cada uno con una sola responsabilidad: MDRS_wrapper (src/MDRS_wrapper.sv) es el DUT; mdrs_if (tb/mdrs_if.sv) son los cables; mdrs_tester (tb/mdrs_tester_pkg.sv) es la clase que sabe el protocolo; mdrs_ref_model (tb/mdrs_ref_model_pkg.sv) es el modelo de referencia que dice qué debe salir. El testbench mdrs_class_tb (tb/mdrs_class_tb.sv) sólo los conecta y llama métodos.

Interfaz mdrs_if

Agrupa todas las señales entre el DUT y el testbench: clk (entrada del interface), rst, load, operation, data, ready, loadX, loadY, result, remainder y error. Tiene dos modports, dut y tb, que marcan la dirección de cada señal desde cada lado. En el testbench se instancia una vez (bus), el MDRS_wrapper se conecta a sus señales y la clase recibe un virtual mdrs_if con el que lee y escribe. La regla de sincronía es la misma de los otros testbenches: el tb escribe en negedge y el DUT muestrea en posedge.

Clase mdrs_tester

Es la única que conoce el protocolo del MDRS. Desde el testbench no se toca ninguna señal.

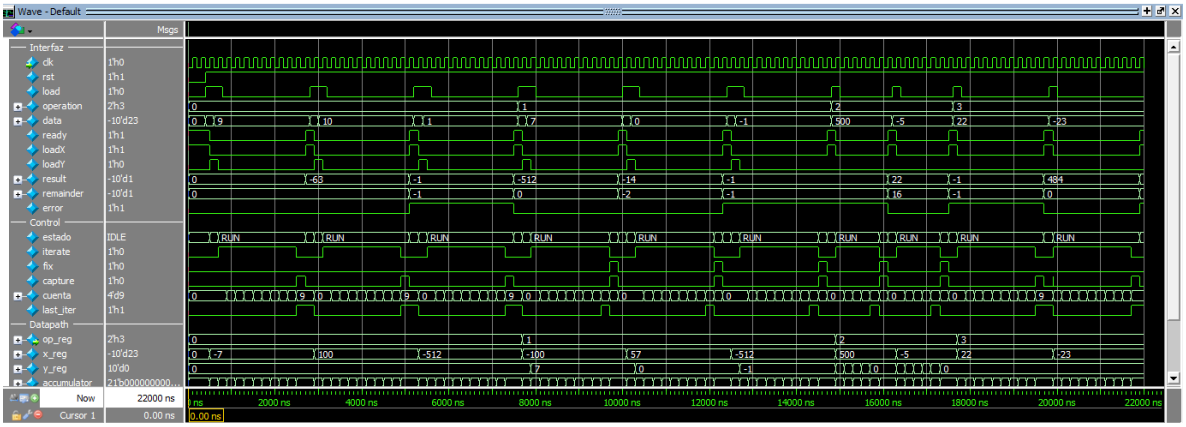
Método	**Qué hace**
reset()	Baja rst tres ciclos y lo suelta; deja load, data y operation en 0.
mult(x, y), div(x, y)	Espera ready y loadX, manda X con load, espera loadY, manda Y con load, espera ready y compara contra el modelo.
sqrt(x), square(x)	Igual pero con un solo operando.
summary()	Imprime PASS y FAIL de cada operación.

Los métodos internos son local: send() hace un load (pone operation y data con load en alto durante un ciclo) y run_op() hace el protocolo completo: espera ready y loadX, manda X, si la operación necesita Y espera loadY y la manda, espera ready y compara contra el modelo de referencia. Si al momento de mandar Y no está loadY o sigue ready, imprime PROTOCOLO. Lleva los contadores pass y fail por operación.

Testbench mdrs_class_tb

- mult: -7 x 9 = -63; 100 x 10 (error, 1000 no cabe); -512 x 1 = -512 (límite negativo).
- div: -100 / 7 = -14 residuo -2; 57 / 0 (error, división entre cero); -512 / -1 (error, 512 no cabe).
- sqrt: 500 = 22 residuo 16; -5 (error, negativo).
- square: 22 = 484; -23 (error, 529 no cabe).

Se corre desde Lab2/sim con do mdrs_class.do. El script compila todo, abre la onda con tres grupos (Interfaz, Control y Datapath) y corre los 10 casos.



Simulación de mdrs_class_tb: los 10 casos dirigidos (grupos Interfaz, Control y Datapath).

```
# OK    MULT   x=-7 y=9 -> r=-63 rem=0 e=0
# OK    MULT   x=100 y=10 -> r=-1 rem=-1 e=1
# OK    MULT   x=-512 y=1 -> r=-512 rem=0 e=0
# OK    DIV    x=-100 y=7 -> r=-14 rem=-2 e=0
# OK    DIV    x=57 y=0 -> r=-1 rem=-1 e=1
# OK    DIV    x=-512 y=-1 -> r=-1 rem=-1 e=1
# OK    SQRT   x=500 y=0 -> r=22 rem=16 e=0
# OK    SQRT   x=-5 y=0 -> r=-1 rem=-1 e=1
# OK    SQUARE x=22 y=0 -> r=484 rem=0 e=0
# OK    SQUARE x=-23 y=0 -> r=-1 rem=-1 e=1
```

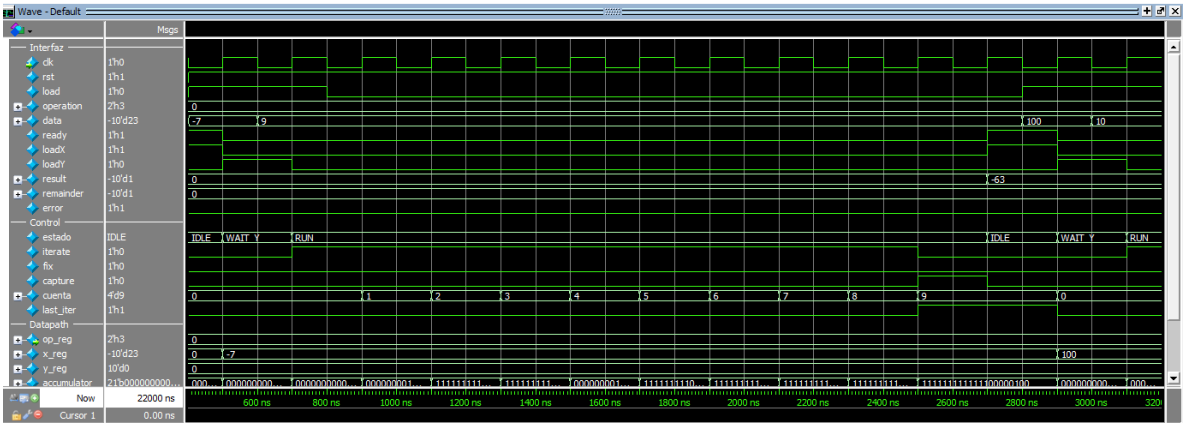
Los diez casos pasan. En la onda se ve que mult, div y square tardan 10 ciclos del último load a ready y sqrt 6, dentro del límite de DW ciclos.

```
# MULT   PASS 3  FAIL 0
# DIV    PASS 3  FAIL 0
# SQRT   PASS 2  FAIL 0
# SQUARE PASS 2  FAIL 0
```

Simulación por operación

Multiplicación -7 × 9.

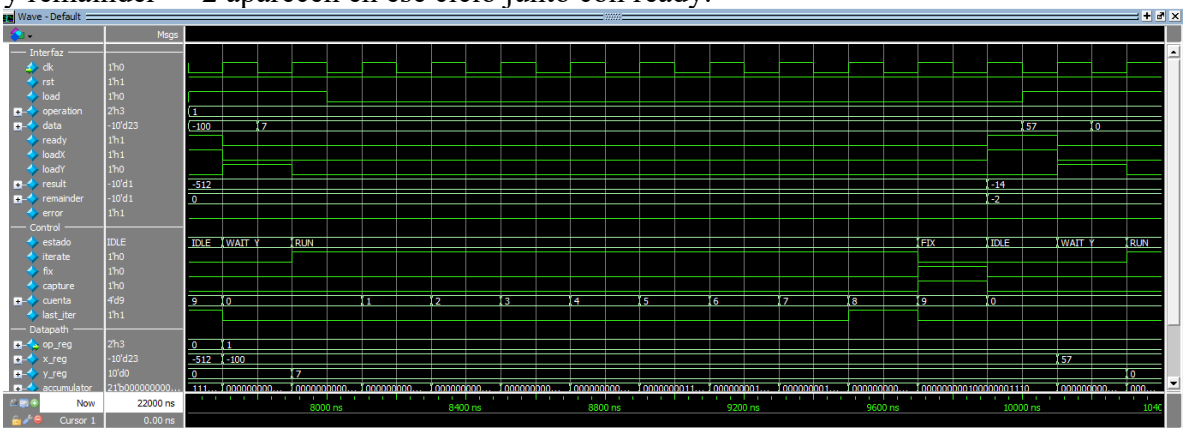
Primer load con data = -7 y operation = 0 estando en IDLE: loadX y ready bajan, la FSM pasa a WAIT_Y y x_reg toma -7. Segundo load con 9: el acumulador se carga con {0, 9, 0} y la FSM pasa a RUN. En RUN iterate está en alto y la cuenta va de 0 a 9; en cada ciclo el acumulador (en binario) decide con sus dos bits bajos si suma, resta o no hace nada con x_reg, y se recorre a la derecha. En el último ciclo sube capture y result toma -63, remainder 0, error 0; ready y loadX vuelven a 1.



Multiplicación -7 × 9: WAIT_Y, 10 iteraciones de Booth en RUN y capture con result = -63.

División -100 / 7.

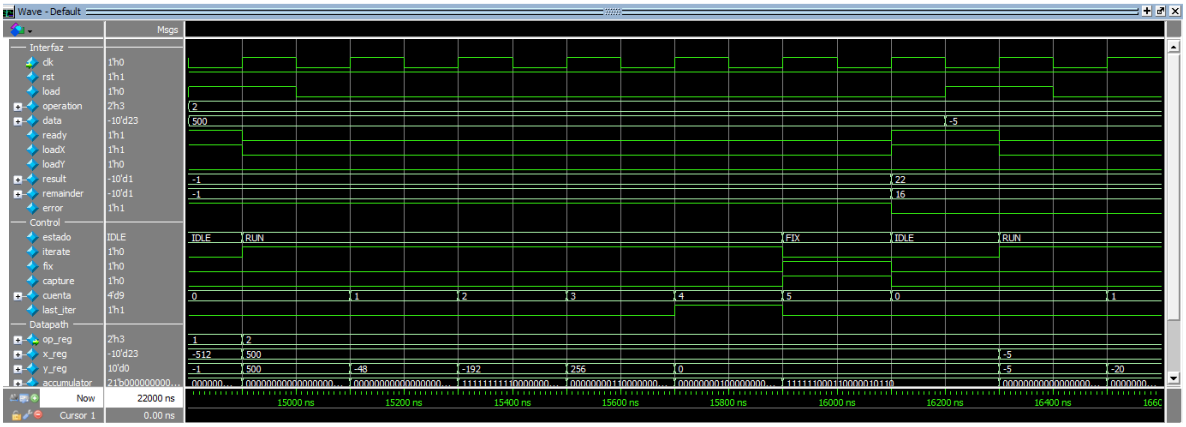
El primer load guarda X = -100 y el acumulador recibe |X| = 100 (el sumador calcula 0 - X con adder_sel = 1). Con el load de Y = 7 se hace la primera iteración y empieza RUN; en cada ciclo adder_a es el residuo parcial corrido un bit, adder_b es 7 y adder_sel = 1 (resta); si la resta no da negativa se guarda y entra un 1 al cociente, si no se conserva el residuo y entra un 0. Tras la iteración 8 (last_iteration) la FSM va a FIX: fix en alto un ciclo, el sumador niega el cociente porque los signos son distintos y el residuo toma el signo de X; result = -14 y remainder = -2 aparecen en ese ciclo junto con ready.



División -100 / 7: iteraciones restoring en RUN, FIX corrige signos, result = -14 y remainder = -2.

Raíz de 500.

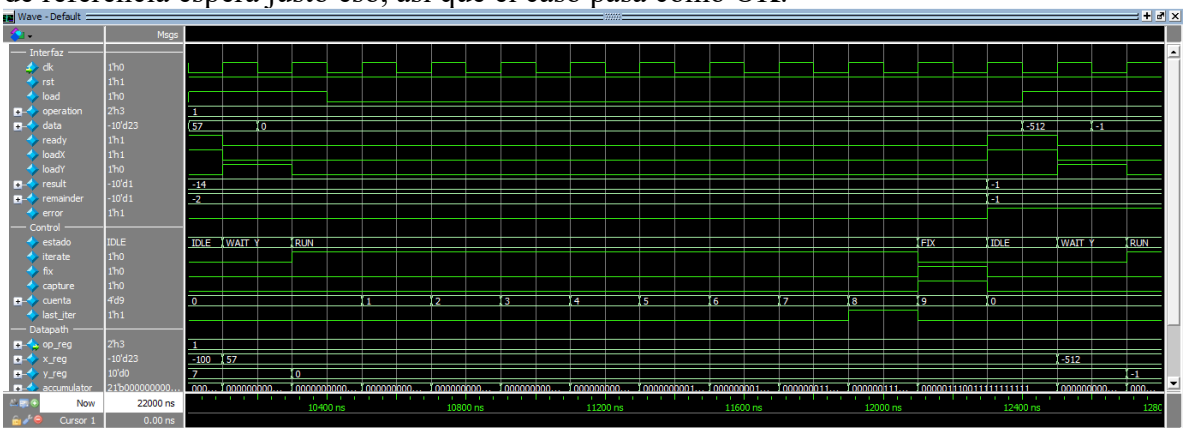
Un solo load: x_reg = 500, y_reg = 500 y acumulador = 0; como Needs_Y = 0 la FSM entra directo a RUN. Son 5 iteraciones (cuenta 0 a 4): y_reg se recorre dos bits a la izquierda por ciclo y sus dos bits altos entran al residuo; adder_b es 4Q + 1 (o 4Q + 3 si el residuo quedó negativo) y adder_sel sigue al signo del residuo. En FIX el residuo quedó positivo, así que no se corrige: result = 22 y remainder = 16 ($22^2 = 484$, 500 - 484 = 16). De load a ready son 6 ciclos.



Raíz de 500: un solo load, 5 iteraciones y FIX; result = 22, remainder = 16.

Error 57 / 0.

La FSM hace exactamente el mismo recorrido que una división normal (no sabe que hay error). En FIX, como y_reg == 0, se escribe error = 1 y result = remainder = -1. El modelo de referencia espera justo eso, así que el caso pasa como OK.



División 57 / 0: mismo recorrido, pero en FIX se escribe error = 1 y result = remainder = -1.

Modelo de referencia (Extra 10%).

El tester no tiene las respuestas escritas a mano: cada caso se califica contra mdrs_ref_model, un modelo en software del MDRS que vive en su propio package (mdrs_ref_model_pkg.sv) para que se pueda reutilizar.

predict(op, x, y) regresa una struct mdrs_expected_t con result, remainder y error. Arranca suponiendo error (result = remainder = -1, error = 1) y sólo lo quita si la operación es válida

Operación	Es error cuando	Si no hay error
mult, square	el producto no cabe en 10 bits con signo (fuera de -512 a 511)	result = producto, remainder = 0
div	y = 0, o x = -512 con y = -1 (512 no cabe)	result = x / y truncado hacia cero, remainder = x % y (signo de x)

sqrt	x < 0	result = mayor q con $q^2 \leq x$, remainder = $x - q^2$

check(e, result, remainder, error) compara lo que sacó el DUT contra la struct. result y remainder se reciben como logic signed de DW bits: sin eso SystemVerilog los extendería con ceros a 32 bits al compararlos con el int del modelo. En run_op, si check regresa 0 se imprime FALLA con lo esperado y lo obtenido.

Implementación.

Mapeo de pines.

Part	Pin	Signal
GPIO		
GPIO[0]	PIN_W15	Pin_tx
GPIO[1]	PIN_AK2	pin_rx
LED		
KEY[0]	PIN_AJ4	Reset
CLK		
CLOCK_50	PIN_AF14	Clock

in Reset	Input	PIN_AJ4
out Pin_tx	Output	PIN_W15
in Pin_rx	Input	PIN_AK2
in Clk	Input	PIN_AF14

Capturas de funcionamiento de MDRS.

Multiplicación

>>> m.load(0, 3);m.load(0, 5)	>>> m.load(0, -5);m.load(0, 5)	>>> m.load(0, 200);m.load(0, 100)
>>> m.load(0, 3);m.load(0, 5) Result = 15 (0x000f) Remainder = 0 (0x0000) LoadX = 0 LoadY = 1 Ready = 0 Error = 0 (15, 0, 0, 1, 0, 0) Result = 15 (0x000f) Remainder = 0 (0x0000) LoadX = 1 LoadY = 0 Ready = 1 Error = 0 (15, 0, 1, 0, 1, 0)	>>> m.load(0, -5);m.load(0, 5) Result = 15 (0x000f) Remainder = 0 (0x0000) LoadX = 0 LoadY = 1 Ready = 0 Error = 0 (15, 0, 0, 1, 0, 0) Result = -25 (0x03e7) Remainder = 0 (0x0000) LoadX = 1 LoadY = 0 Ready = 1 Error = 0 (-25, 0, 1, 0, 1, 0)	>>> m.load(0, 200);m.load(0, 100) Result = -25 (0x03e7) Remainder = 0 (0x0000) LoadX = 0 LoadY = 1 Ready = 0 Error = 0 (-25, 0, 0, 1, 0, 0) Result = -1 (0x03ff) Remainder = -1 (0x03ff) LoadX = 1 LoadY = 0 Ready = 1 Error = 1 (-1, -1, 1, 0, 1, 1)

División

>>> m.load(1, 4);m.load(1, 100)

```
>>> m.load(1, 4);m.load(1, 100)
Result = 0 (0x0000)
Remainder =  4 (0x0004)
LoadX = 0
LoadY = 1
Ready = 0
Error = 0
(0, 4, 0, 1, 0, 0)
Result = 0 (0x0000)
Remainder =  4 (0x0004)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(0, 4, 1, 0, 1, 0)
```

>>> m.load(1, 47);m.load(1, 0)

```
>>> m.load(1, 47);m.load(1, 0)
Result = 0 (0x0000)
Remainder =  4 (0x0004)
LoadX = 0
LoadY = 1
Ready = 0
Error = 0
(0, 4, 0, 1, 0, 0)
Result = -1 (0x03ff)
Remainder =  -1 (0x03ff)
LoadX = 1
LoadY = 0
Ready = 1
Error = 1
(-1, -1, 1, 0, 1, 1)
```

>>> m.load(1, 47);m.load(1, 2)

```
>>> m.load(1, 47);m.load(1, 2)
Result = -1 (0x03ff)
Remainder =  -1 (0x03ff)
LoadX = 0
LoadY = 1
Ready = 0
Error = 1
(-1, -1, 0, 1, 0, 1)
Result = 23 (0x0017)
Remainder =  1 (0x0001)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(23, 1, 1, 0, 1, 0)
```

>>> m.load(1, -125);m.load(1, 5)

```
>>> m.load(1, -125);m.load(1, 5)
Result = 23 (0x0017)
Remainder =  1 (0x0001)
LoadX = 0
LoadY = 1
Ready = 0
Error = 0
(23, 1, 0, 1, 0, 0)
Result = -25 (0x03e7)
Remainder =  0 (0x0000)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(-25, 0, 1, 0, 1, 0)
```

Raíz

>>> m.load(2, -91)

```
Result = -1 (0x03ff)
Remainder =  -1 (0x03ff)
LoadX = 1
LoadY = 0
Ready = 1
Error = 1
(-1, -1, 1, 0, 1, 1)
```

>>> m.load(2, 89)

```
Result = 9 (0x0009)
Remainder =  8 (0x0008)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(9, 8, 1, 0, 1, 0)
```

>>> m.load(2, 9)

```
Result = 3 (0x0003)
Remainder =  0 (0x0000)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(3, 0, 1, 0, 1, 0)
```

Square

>>> m.load(3, 9)

```
Result = 81 (0x0051)
Remainder =  0 (0x0000)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(81, 0, 1, 0, 1, 0)
```

>>> m.load(3, 100)

```
Result = -1 (0x03ff)
Remainder =  -1 (0x03ff)
LoadX = 1
LoadY = 0
Ready = 1
Error = 1
(-1, -1, 1, 0, 1, 1)
```

>>> m.load(3, -8)

```
Result = 64 (0x0040)
Remainder =  0 (0x0000)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(64, 0, 1, 0, 1, 0)
```

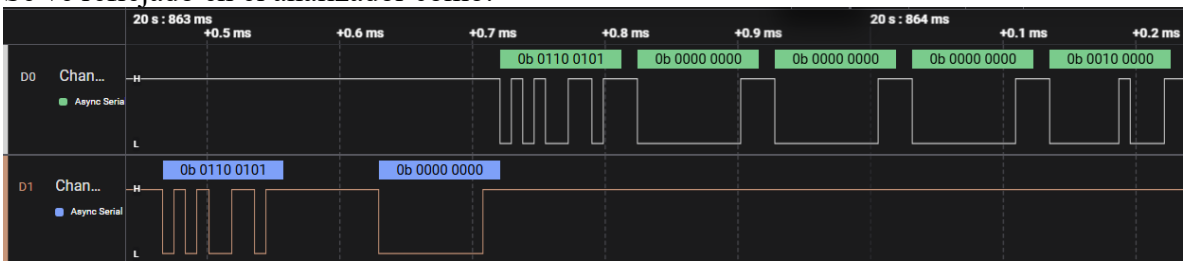
Validación.

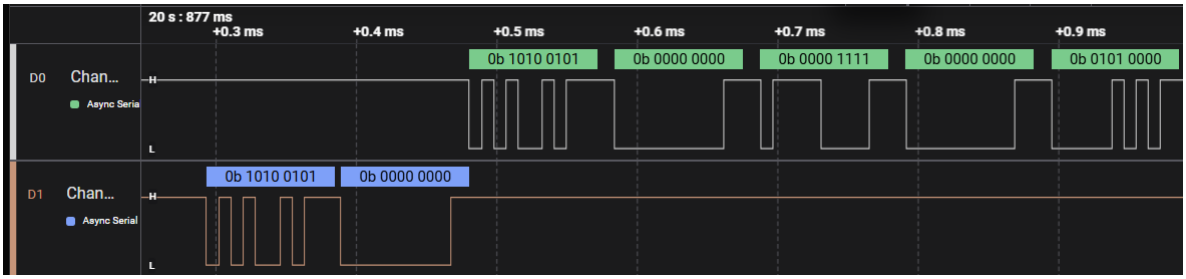
Tomando en cuenta en la estructura en la que se mandan los datos se tiene esto.

BYTE	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
TX_0	data_lower_3_bits			operation		Load	Rsvd	Trigger=1
TX_1	data_higher_7_bits							0
RX_0	data_lower_3_bits			operation		Load	Rsvd	1
RX_1	data_higher_7_bits							0
RX_2	result_lower_8_bits							
RX_3	remainder_lower_6_bits						result_higher_2_bits	
RX_4	error	ready	loadY	loadX	remainder_higher_4_bits			

Enviando un: m.load(0, 3);m.load(0, 5)

Se ve reflejado en el analizador como:





Por lo tanto visto en la estructura:

m.load(0, 3)

	Bit							
BYTE	7	6	5	4	3	2	1	0
Tx_0	0	1	1	0	0	1	0	1
Tx_1	0	0	0	0	0	0	0	0
Rx_0	0	1	1	0	0	1	0	1
Rx_1	0	0	0	0	0	0	0	0
Rx_2	0	0	0	0	0	0	0	0
Rx_3	0	0	0	0	0	0	0	0
Rx_4	0	0	1	0	0	0	0	0

m.load(0, 5)

	Bit							
BYTE	7	6	5	4	3	2	1	0
Tx_0	1	0	1	0	0	1	0	1
Tx_1	0	0	0	0	0	0	0	0
Rx_0	1	0	1	0	0	1	0	1
Rx_1	0	0	0	0	0	0	0	0
Rx_2	0	0	0	0	1	1	1	1
Rx_3	0	0	0	0	0	0	0	0
Rx_4	0	1	0	1	0	0	0	0

Con m.load(0, 3):
Tx_0: data_lower_3_bits = 011, operation = 00 (mult), Load = 1, Rsvd = 0, Trigger = 1
Tx_1: data_lower_7_bits = 0000 000, y el siempre 0
Rx_0: data_lower_3_bits = 011, operation = 00 (mult), Load = 1, Rsvd = 0, el siempre 1
Rx_1: data_lower_7_bits = 0000 000, y el siempre 0
Rx_2: result_lower_8_bits = 0000 0000
Rx_3: remainder_lower_6_bits = 00 0000, result_higher_2_bits = 00
Rx_4: error = 0, ready = 0, loadY = 1, loadX = 0, remainder_higher_4_bits = 0000

Con m.load(0, 5):
Tx_0: data_lower_3_bits = 101, operation = 00 (mult), Load = 1, Rsvd = 0, Trigger = 1
Tx_1: data_lower_7_bits = 0000 000, y el siempre 0
Rx_0: data_lower_3_bits = 101, operation = 00 (mult), Load = 1, Rsvd = 0, el siempre 1
Rx_1: data_lower_7_bits = 0000 000, y el siempre 0
Rx_2: result_lower_8_bits = 0000 1111
Rx_3: remainder_lower_6_bits = 00 0000, result_higher_2_bits = 00
Rx_4: error = 0, ready = 1, loadY = 0, loadX = 1, remainder_higher_4_bits = 0000

Y ya en la terminal esta:
Result = 0 (0x0000)
Remainder = 0 (0x0000)
LoadX = 0
LoadY = 1
Ready = 0
Error = 0
(0, 0, 0, 1, 0, 0)
Result = 15 (0x000f)
Remainder = 0 (0x0000)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(15, 0, 1, 0, 1, 0)

```
>>> m.load(0, 3);m.load(0, 5)
Result = 0 (0x0000)
Remainder =  0 (0x0000)
LoadX = 0
LoadY = 1
Ready = 0
Error = 0
(0, 0, 0, 1, 0, 0)
Result = 15 (0x000f)
Remainder =  0 (0x0000)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(15, 0, 1, 0, 1, 0)
```

Python Script auto-check (Extra 10%).

Se mantuvo el driver original (115200 baudios, paridad impar, 2 bytes de envío y 5 de respuesta). Se agregó una función de verificación automática contra un modelo de referencia en software.

Los cambios principales son.

wait_bytes() en la clase UART, llamado en _send_receive_cmd.
Espera a que lleguen los 5 bytes, sin la pausa fija de 1 s por comando. La automatización pasa de horas a minutos.

Parámetro quiet en load() y _send_receive_cmd.
Permite silenciar los "printf" durante la automatización. Asi se evita que imprima todos los casos cuando se prueben los todos los casos posibles.

load() retorna (result, remainder, loadX, loadY, ready, error)
El resultado se puede usar desde código y no solo leerlo en el log.

Se agregaron 2 import nuevas.

math	isqrt para la raíz cuadrada entera del modelo
random	Casos aleatorios y mezcla de operaciones

Se agrego también una función nueva. auto(mode=X, n=Y).

Para auto se tienen los siguientes argumentos posibles.

Modo	Que hace
0	Multiplicacion
1	Division
2	Square Root
3	Cuadrado
4	Todas las operaciones que deberían pasar
5	Todas las operaciones que no deberían pasar

N	Efecto
0	Todos los casos del modo seleccionado
>0	n casos al random por operación

Datapath estructural.

Para el extra se hizo una segunda versión del datapath, "MDRS_datapath_estructural", que es la que va en el top. Hace exactamente lo mismo que "MDRS_datapath" pero sin ningún always_comb ni always_ff: todo son instancias de bloques básicos y cables (assign con concatenaciones y compuertas).

Bloques básicos

- register_ar: registro con enable y reset asíncrono activo en 0. Es el único bloque secuencial.

- mux2: multiplexor 2 a 1 de DW bits.

- mux4: multiplexor 4 a 1, armado con tres mux2.

- Sumador_Restador: el mismo de siempre, es la única operación aritmética.

Estos cuatro caen en la lista del spec (registros, multiplexores y operaciones add/sub); las compuertas van como assign con &, |, ^ y ~.

Cómo se pasó de conductual a estructural

La idea fue que el estructural se leyera igual que el conductual: mismas señales, mismo orden. Cada construcción se tradujo así:

Conductual	**Estructural**
if (c) x = a; else x = b;	mux2 con sel = c
case (op_now) con DIV, SQRT y default	mux4 con sel = op_now (in0 e in3 son Booth, igual que el default)
if / else if / else de los operandos del sumador por fase	dos mux2 anidados
case (Product[1:0]) de Booth	compuertas: Accumulate_w = P[1] ^ P[0], Subtract_w = P[1] & ~P[0]
(y_reg == 0), &bits, |bits	compuertas de reducción (~|, &, |)
always_ff con case (op_now)	un cable *_next por registro armado con muxes y un register_ar con su enable
Concatenaciones y rebanadas ({a, b}, acc[9:0])	se quedan igual, son solo cables

Decisiones que se tomaron:

- Los enables salen tal cual del always_ff original: “op_reg” y “x_reg” con save_x; “y_reg” con save_y en división y con save_x o en_step en raíz; “accumulator” con save_x, save_y o en_step; result, remainder y error con capture o en_write_result (la FSM los sube juntos en FIX).

- Cuando no hay ninguna fase activa el sumador calcula el caso de FIX. Como en ese ciclo ningún registro tiene enable no afecta, y se ahorran tres mux de 21 bits.

Resultados

Los 64 flip-flops quedan repartidos en los 7 register_ar y hay un solo Sumador_Restador, que es lo que se ve en el RTL Viewer.

Registro	**Flip-flops**	**Máximo del spec**
accumulator_register	21	2*DW + 1 = 21
x_register	10	DW = 10
y_register	10	DW = 10
result_register	10	DW = 10
remainder_register	10	DW = 10
op_register	2	2
error_register	1	1
Total	64	65 (ready esta en la FSM)

Referencias.

[1] W. Green, "Division in Verilog," Project F, Jun. 2021. Accedido: Oct. 26, 2026. [En línea]. Disponible: https://projectf.io/posts/division-in-verilog/
[2] W. Green, "Square Root in Verilog," Project F, Jun. 27, 2021. Accedido: Oct. 26, 2026. [En línea]. Disponible: https://projectf.io/posts/square-root-in-verilog/
[3] Intel, "Signal Tap Logic Analyzer: Introduction & Getting Started," Quartus Prime Pro/Standard Edition User Guide: Debug Tools. [En línea]. Disponible: https://www.intel.com/content/www/us/en/docs/programmable/683819/
[4] IEEE Std 1800-2017, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language. (interfaces, clases, virtual interface, $signed).

Anexo. Prompts de IA utilizados.

Durante la práctica se usó un asistente de IA Claude sonnet como apoyo para entender algoritmos, revisar módulos y organizar el trabajo. Se incluyen los prompts y las respuestas que se usaron.

Prompt 1.

https://projectf.io/posts/division-in-verilog/
Revisa a detalle esta pagina y dime si el algoritmo sirve como base para el desarrollo de división restoring en system verilog

Sí, la página de Project F usa el mismo algoritmo que tu divisor: división larga en binario de desplazar y restar (restoring, un ciclo por bit). Sirve para entender el algoritmo y para citarla en el reporte, pero no conviene copiar su forma de implementarlo, porque se brinca varias restricciones del lab.

En qué es diferente

Project F	**Tu divisor (por las reglas del lab)**
Compara acc >= bu y además resta acc - bu: son dos operaciones aritméticas	Una sola resta; su bit de signo dice si cupo
Calcula -a, -b y al final -quo: más restadores	Usa **un solo sumador** para todo; |Y| nunca se calcula y las dos negaciones finales salen en una sola suma
Marca overflow si **cualquier** operando es el número más negativo (−512 en 10 bits), porque sus magnitudes son 1 bit más angostas	Acepta −512; solo −512 / −1 da error
La versión con signo solo entrega el cociente, sin residuo	Entrega el cociente y el residuo con signo

Project F	Tu divisor (por las reglas del lab)
Usa start/busy/done/valid y recibe a y b al mismo tiempo	Usa load/loadX/loadY/ready/error y un solo puerto data
Reset síncrono, activo en alto	Reset asíncrono, activo en bajo
Además maneja punto fijo y redondeo	No hace falta en este lab
Toma varios ciclos de más (INIT, ROUND, SIGN)	ready sube a los 10 ciclos de cargar Y, como exige la spec

Si escribieras el divisor como Project F (con >=, -a, -quo), en el RTL Viewer aparecerían varios sumadores y la calificación máxima bajaría a 70.

Prompt 2.

Quiero que me expliques como niño chiquito el algoritmo que estoy implementando de la division restoring, paso a paso. muy bien explicado

Dividir en binario es preguntar muchas veces "¿cabe o no cabe?" y anotar un 1 o un 0 cada vez.
1. ¿Qué es dividir?
Tienes 7 dulces y 2 amigos, y a cada uno le das lo mismo. A cada quien le tocan 3 y te sobra 1.

Nombre	En el ejemplo	Qué es
Dividendo (X)	7	Lo que repartes
Divisor (Y)	2	Entre cuántos repartes
Cociente	3	Cuánto le toca a cada uno
Residuo	1	Lo que sobra

Siempre se cumple: dividendo = cociente × divisor + residuo. Aquí: 7 = 3 × 2 + 1.
2. Cómo divides a mano (en decimal)
Por ejemplo, 74 ÷ 3:

1. Bajas la primera cifra, el 7. El 3 cabe 2 veces en el 7 (2 × 3 = 6) y sobra 1.
2. Bajas la siguiente cifra, el 4, y la pegas al 1 que sobró: queda 14. El 3 cabe 4 veces en el 14 (4 × 3 = 12) y sobra 2.
3. Ya no quedan cifras: el cociente es 24 y el residuo 2. Comprobación: 24 × 3 + 2 = 74.

Solo repites dos cosas: bajar una cifra y ver cuántas veces cabe.
3. En binario es más fácil
En binario cada cifra del cociente solo puede ser 0 o 1. Así que la pregunta "¿cuántas veces cabe?" se convierte en "¿cabe o no cabe?", y se responde restando:

- Si la resta da 0 o más, sí cabe. Anotas un 1 y te quedas con lo que sobró de la resta.

- Si la resta da negativo, no cabe. Anotas un 0 y deshaces la resta, es decir, vuelves al número que tenías antes.

Ese "deshacer" es lo que significa *restoring* (restaurar).
4. Las tres cajitas
En hardware usas tres cajitas (registros):

- R (residuo): lo que va sobrando. Empieza en 0.
- Q (dividendo que se vuelve cociente): empieza con X. Funciona como una banda transportadora: por la izquierda van saliendo las cifras de X para "bajarlas", y por la derecha van entrando las cifras de la respuesta.
- D (divisor): es Y y nunca cambia.

5. La receta: una vuelta por cada bit

1. Baja la siguiente cifra: recorre R un lugar a la izquierda y pégale el bit que sale por la izquierda de Q. Eso es lo mismo que calcular R × 2 + ese bit.
2. Prueba: réstale D.
3. Revisa si salió negativo:
    - Si no salió negativo, cabe: R se queda con el resultado de la resta y el bit nuevo es 1.
    - Si sí salió negativo, no cabe: R se queda como estaba antes de restar y el bit nuevo es 0.
4. Mete el bit nuevo por la derecha de Q.

Con 4 bits das 4 vueltas; con 10 bits, 10 vueltas.
6. Ejemplo completo: 11 ÷ 3 con 4 bits
X = 11 = 1011 y D = 3 = 0011. En la columna Q, la barra separa lo que falta del dividendo (a la izquierda) de las cifras del cociente ya escritas (a la derecha).

Vuelta	Bit que bajas	R × 2 + bit	Menos 3	¿Cabe?	R queda	Q
inicio					0	1011|
1	1	0×2 + 1 = 1	−2	no → 0	1	011|0
2	0	1×2 + 0 = 2	−1	no → 0	2	11|00
3	1	2×2 + 1 = 5	2	sí → 1	2	1|001
4	1	2×2 + 1 = 5	2	sí → 1	2	|0011

Al final, Q = 0011 = 3 y R = 2. Comprobación: 3 × 3 + 2 = 11.
Fíjate cómo en cada vuelta sale un bit de X por la izquierda y entra un bit de la respuesta por la derecha. Después de 4 vueltas, toda la cajita Q ya es la respuesta.
7. ¿Y los números negativos?
El truco es dividir como si todo fuera positivo y al final ponerle el signo:

- El cociente es negativo si X y Y tienen signos diferentes; eso es lo que calcula el XOR.
- El residuo lleva el signo de X.
- En complemento a 2, volver negativo un número es invertir todos sus bits y sumarle 1 (~N + 1).

Ejemplo, −7 ÷ 2:

1. Sin signos: 7 ÷ 2 = 3 y sobra 1.
2. X y Y tienen signos diferentes, así que el cociente es −3.
3. X era negativo, así que el residuo es −1.
4. Comprobación: (−3) × 2 + (−1) = −7.

Tu diseño tiene además un atajo: si Y es negativo, no hace falta calcular |Y|, porque restar |Y| es lo mismo que sumar Y. Por ejemplo, restar 3 da lo mismo que sumar −3.

8. Cuando no se puede dividir

- División entre 0: no tiene respuesta. Se pone error = 1 y result = remainder = −1.
- −512 ÷ −1 = +512: el resultado no cabe en 10 bits, porque el máximo es 511. Se marca el mismo error.

Y así se reparte en el tiempo:

1. Ciclo en que llega X: se calcula |X|.
2. Ciclo en que llega Y: se hace la vuelta 1.
3. Los 9 ciclos siguientes: vueltas 2 a 10.
4. Último ciclo: se ponen los signos y sube ready.

Prompt 3.

Revisa exhaustivamente el modulo de booth_datapath y revisa como se podrían hacer correcciones para que funcione correctamente para el lab 2
El Booth está bien hecho: lo probé con todas las combinaciones posibles y no falla ninguna. Lo que no cumple son las restricciones del Lab 2, y al ajustarlo aparecen exactamente 2 casos que hay que resolver.

Versión	**Correctos**	**Fallas**
Actual (acumulador de 22 bits)	1,049,600	**0**
Con acumulador de 21 bits (lo que pide la spec)	1,049,598	**2**: −512 × 1 y −512 × −1

Lo que está bien y se queda igual

- La decisión de Booth con Product[1:0].

- El corrimiento aritmético: funciona porque Product_pre_shift_w es signed.
- La carga inicial del producto.
- La detección de overflow en DMRS_result.
- Los tiempos: ready sube a los 10 ciclos de cargar Y, el máximo que permite la spec.

Lo que hay que corregir para el Lab 2

1. Acumulador de 22 → 21 bits. El campo A pasa de DW+1 a **DW** bits, como en el pseudocódigo de la spec.

2. Multiplicando de 11 → 10 bits. Como A ahora mide DW bits, ya no hace falta extender el signo. X se guarda tal cual en un registro de 10 bits.

3. Los 2 casos con −512.

- Con A de 10 bits, restar un multiplicando de −512 da +512, que no cabe. El signo sale mal y el producto también. Por eso Victor usó un bit extra en el Lab 1.
- Solo afecta a dos casos:
  - −512 × 1 debería dar −512, pero sale error.
  - −512 × −1 debería dar error (512 no cabe), pero sale −512 sin error.
- Las demás multiplicaciones con −512 no caben en 10 bits, y el error se sigue detectando bien. La simulación lo confirma: hay 0 fallas con X distinto de −512.
- Opciones:
  - **Recomendada: intercambiar los papeles cuando X = −512.** Al capturar Y, si X es −512, el producto se carga con X como multiplicador y el registro de X con Y como multiplicando. Así el multiplicando es ±1 y A no se desborda; como la multiplicación es conmutativa, el resultado es el mismo. Esta opción no la he simulado; si quieres, la verifico.
  - **Alternativa: preguntarle al profe si acepta 22 bits.** El pseudocódigo de la spec usa enteros de Python, que no tienen ancho fijo, así que este caso no aparece ahí.

4. Para integrarlo al datapath compartido. Booth no debe tener su propio Sumador_Restador. El bloque booth_step debe sacar a = Accumulator_w, b = Multiplicand_r y sel = Subtract_w, y recibir el resultado del sumador compartido. El mux de Accumulate_w se queda igual.

Tu archivo actual **todavía falla**. Con los cambios de abajo pasan **todos**, ya con producto de 21 bits. Además ya no hace falta intercambiar X y Y para el caso −512.
Lo que ya ajustaste para los 21 bits está bien: los anchos de los puertos y la carga inicial.

1. Ancho del sumador

- El sumador está instanciado con .DW(DW+1), pero le conectas señales de DW bits.
- Fallan casi todos los productos que sí caben en 10 bits.

La solución es extender el signo a mano y que el resultado también tenga DW+1 bits:
Después borra Product_pre_shift_w y el >>>.

Por qué esto también resuelve el −512:

- A ± M puede necesitar 11 bits; por ejemplo, 0 − (−512) = +512.
- Al recorrer a la derecha, ese bit extra pasa a ser el nuevo signo de A, y el bit 0 de A pasa a Q. Así, {A±M (11), Q (10)} da exactamente 21 bits.
- Solo se descarta el Q_{-1} viejo, que ya no se necesita, así que no se pierde nada.

2. Puertos que sobran

- Multiplicand_extension_w está declarada como salida pero nadie la asigna, así que vale X.
- Multiplicand_init ya no se usa.

Borra los dos. En el top, el registro de X (10 bits) toma mdrs_data_w directo.
Cuando lo integres con el sumador compartido de 21 bits, la idea es la misma: extiende el signo de A y M a 21 bits y toma los bits del resultado.

Prompt 4.

Quiero hacer la versión estructural del datapath. Tiene que hacer exactamente lo mismo que mi MDRS_datapath (mismas señales, mismo orden, mismo comportamiento ciclo a ciclo) pero sin ningún always: solo registros, muxes, compuertas y el Sumador_Restador. Dime cómo traducirías cada parte, qué hay que cuidar para que siga siendo un solo sumador y 64 flip-flops, y cómo compruebo que las dos versiones son idénticas.

La idea es no rediseñar nada. El conductual ya pasó el barrido exhaustivo, así que el estructural se arma traduciendo construcción por construcción, dejando los mismos nombres de señal y el mismo orden en el archivo. Si al final las dos versiones se leen igual, cualquier diferencia de comportamiento se encuentra comparando renglón contra renglón.

Bloques básicos

Solo hacen falta cuatro, y los cuatro caen en la lista que permite el spec (registros, multiplexores y operaciones add/sub):

- register_ar: registro de DW bits con enable y reset asíncrono activo en 0. Es el único bloque secuencial de todo el datapath; la terminación "ar" es por asynchronous reset.
- mux2: multiplexor 2 a 1 de DW bits con sel, in1 e in0. Es el bloque que más se repite.
- mux4: multiplexor 4 a 1 armado con tres mux2 (dos para la primera etapa y uno para la segunda). Se usa donde el conductual tenía un case por operación.
- Sumador_Restador: el mismo de siempre, es la única operación aritmética del datapath. No se instancia otro.
- Las compuertas no necesitan módulo: van como assign con &, |, ^ y ~, igual que las concatenaciones y las rebanadas, que son solo cables.

Cómo se traduce cada construcción

- if (c) x = a; else x = b; se vuelve un mux2 con sel = c, in1 = a, in0 = b. Si el if no tiene else y la señal es un registro, la condición es el enable del register_ar y no hace falta mux.
- case (op_now) con DIV, SQRT y default se vuelve un mux4 con sel = op_now; las entradas in0 e in3 reciben lo mismo que el default (Booth), porque mult y square son los códigos 0 y 3.
- Los if / else if / else que eligen los operandos del sumador por fase (save_x, Booth, restoring, raíz, FIX) se vuelven dos mux2 anidados: el primero decide entre fase de carga y fase de operación, el segundo entre las operaciones.
- El case de Booth se vuelve dos compuertas: Accumulate = P[1] ^ P[0] (los bits son distintos, hay que sumar o restar) y Subtract = P[1] & ~P[0] (el caso 10 resta). Con eso el mux de acumulador queda con sel = Accumulate y el sel del sumador es Subtract.
- Las comparaciones (y_reg == 0), &bits y |bits se vuelven compuertas de reducción: ~|y_reg, &bits, |bits.
- Cada always_ff se vuelve un cable *_next armado con muxes más un register_ar con su enable. El contenido del always_ff dice qué va en el cable; la condición del if dice qué va en el enable.

Lo que hay que cuidar

- Un registro por always_ff y nada más. Los enables salen tal cual del código original: op_reg y x_reg con save_x; y_reg con save_y en división y con save_x o en_step en raíz (porque ahí hace de registro de corrimiento); accumulator con save_x, save_y o en_step; result, remainder y error con capture o en_write_result, que la FSM sube juntos en FIX. Así los 64 flip-flops quedan repartidos en siete register_ar (21 + 10 + 10 + 10 + 10 + 2 + 1) y el Fitter no reporta ninguno extra.
- Un solo sumador. Todas las fases tienen que llegar a adder_a, adder_b y adder_sel por los muxes. Si en algún lado se escribe a + b, a - b o ~a + 1 fuera del Sumador_Restador, Quartus sintetiza otro sumador, el RTL Viewer lo muestra y se pierde el extra. Las negaciones de FIX (cociente y residuo) también pasan por el único sumador, por eso FIX es un ciclo aparte.
- Qué calcula el sumador cuando no hay fase activa. En lugar de poner ceros, conviene que los muxes dejen el caso de FIX como default: en ese ciclo ningún registro tiene enable, así que el valor no se guarda, y se ahorran tres mux de 21 bits.
- op_now. En el ciclo de save_x la operación todavía no está en op_reg (se escribe en ese mismo flanco), así que el mux que elige la fase tiene que usar. Es un mux2 de 2 bits y es el mismo truco del conductual; si se olvida, la primera iteración de división y raíz usa la operación anterior.
- Signo extendido a mano. El sumador compartido es de 21 bits; los operandos de Booth (A de DW bits, M de DW bits) se extienden en signo con concatenación

{{(DW+1){A[DW-1]}}, A} y del resultado se toman los bits [DW:0]. Eso deja el bit extra que resuelve el caso -512 sin agrandar el acumulador.

- Anchos y nombres. Mantener los mismos nombres de señal (adder_a, adder_b, adder_sel, adder_sum, Product_next, shifted_remainder, root_bit) permite que el mismo script de ondas (.do) sirva para las dos versiones y que la comparación en el tb sea señal por señal.

Cómo comprobar que las dos versiones son idénticas

- Un testbench que instancia los dos datapaths con una sola FSM y un solo contador, de modo que reciben exactamente los mismos estímulos en el mismo ciclo.
- En cada negedge compara result, remainder, error y op_reg de las dos versiones; si algo difiere imprime el tiempo y los dos valores. Esto atrapa diferencias de un ciclo que una comparación solo al final no vería.
- Casos dirigidos primero (una operación normal y los errores de cada una), luego sqrt y square con los 1024 valores, luego todas las parejas de valores límite para mult y div (-512, -511, -256, -1, 0, 1, 255, 256, 511...) y aleatorios.
- El mismo barrido exhaustivo que ya pasó el conductual: todas las multiplicaciones y divisiones (1024 x 1024 cada una). Tarda unos 25 minutos.
- Si salen 0 diferencias y 0 fallas, el estructural se pone en el top en lugar del conductual sin tocar nada más, porque los puertos son los mismos; después se vuelve a simular el top por UART y se compila para ver el RTL Viewer y el conteo de registros.

Resultado

- 0 diferencias ciclo a ciclo entre conductual y estructural en 2.3 millones de operaciones, 0 fallas contra el esperado.
- 64 flip-flops en el datapath (el spec permite 65; ready vive en la FSM).
- Un solo Sumador_Restador en el RTL Viewer; todo lo demás son register_ar, mux2, mux4 y compuertas.
- La simulación del top por UART y las pruebas en la tarjeta con el driver de Python dieron los mismos resultados que con el conductual.

Prompt 5.

La práctica pide que el testbench use una interfaz para conectar el MDRS y al menos una clase que lo maneje con métodos de alto nivel, y como extra un modelo de referencia. Explícame cómo lo harías partiendo de mi mdrs_tb (que ya tiene las tareas do_op y expected).

Tu mdrs_tb ya tiene las piezas; lo que falta es repartirlas en archivos con una sola responsabilidad cada uno. La tarea do_op es lo que va a ser la clase, la función expected es el modelo de referencia y las señales sueltas que hoy declaras en el módulo son la interfaz. El testbench final solo conecta y llama métodos.

Los archivos y de dónde sale cada uno

-MDRS_wrapper.sv: el DUT. Envuelve UART_MDRS_FSM, Iteration_counter y MDRS_datapath_estructural con los puertos de la práctica (clk, rst, load, operation, data, ready, loadX, loadY, result, remainder, error). Adentro son las mismas conexiones que hay en UART_DMRS, nada más sin el UART. Así lo que se simula es exactamente lo que va en la FPGA.

- tb/mdrs_if.sv: la interfaz. Sale de las señales que mdrs_tb declaraba sueltas.

- tb/mdrs_tester_pkg.sv: la clase. Sale de la tarea do_op.
- tb/mdrs_ref_model_pkg.sv: el modelo de referencia. Sale de la función expected.
- tb/mdrs_class_tb.sv: el testbench. Sale del initial de mdrs_tb, pero sin tocar señales.

Interfaz

- Es un interface que declara rst, load, operation, data, ready, loadX, loadY, result, remainder y error. El reloj entra como puerto del interface para que todos lo compartan.
- Tiene dos modports, dut y tb, que marcan la dirección de cada señal vista desde cada lado. Sirven de documentación y para que el compilador se queje si alguien escribe una salida del DUT desde el tb.
- En el testbench se instancia una sola vez (bus) y el MDRS_wrapper se conecta con bus.clk, bus.rst, bus.load, etc. La clase recibe bus en el constructor y lo guarda en un vif. Un virtual interface es un apuntador a la instancia real: cuando la clase hace vif.load = 1 está moviendo la misma señal que ve el DUT.
- La regla de sincronía es la misma de tus otros tb: la clase escribe en negedge y el DUT muestrea en posedge, así no hay carreras y la onda se lee fácil.

Clase

- Es la única que conoce el protocolo. Los métodos públicos son reset(), mult(x, y), div(x, y), sqrt(x), square(x) y summary(); los cuatro de operación son una línea cada uno: llaman a run_op(op, x, y) con el código correspondiente del package MDRS_pkg (OP_MULT, OP_DIV, OP_SQRT, OP_SQUARE).
- reset() baja rst tres ciclos y lo suelta, dejando load, data y operation en cero.
- send(op, value) es local y hace un load: pone data y operation con load en alto durante un ciclo y al siguiente baja load. Es tu do_op recortado a lo mínimo.
- run_op(op, x, y) es local y hace el protocolo completo: primero pide al modelo el resultado esperado; espera a que ready y loadX estén en 1; manda X con send(); si la operación es mult o div comprueba que loadY esté en 1 y ready en 0 y manda Y; espera a que ready suba; y entonces llama a check del modelo. Si pasa, incrementa pass[op] e imprime OK con los valores; si no, incrementa fail[op] e imprime FALLA con lo esperado y lo obtenido.
- Lleva los contadores pass y fail por operación y el arreglo name con los nombres para los prints; los dos son los mismos de tu mdrs_tb. summary() imprime una línea por operación con PASS y FAIL.

Modelo de referencia

- Va en su propio package para que se pueda reutilizar desde otro tb. Declara una struct mdrs_expected_t con result, remainder (int) y error (bit).
- predict(op, x, y) regresa esa struct. Arranca suponiendo error (result = remainder = -1, error = 1) y solo lo quita cuando la operación es válida: en mult y square si el producto cabe entre -512 y 511 (result = producto, remainder = 0); en div si y no es 0 y no es el caso -512 / -1 (result = x / y truncado hacia cero, remainder = x % y, que toma el signo de x); en sqrt si x no es negativo (result = el mayor q con q*q <= x, remainder = x - q*q). Son las mismas reglas que tenía tu expected y las mismas que usa el script de Python en _expected(), así que simulación y hardware se califican igual.
- check(e, result, remainder, error) compara lo que sacó el DUT contra la struct y regresa 1 o 0.
- Un detalle que sí importa: check debe recibir result y remainder como logic signed [DW-1:0] (o comparar con $signed). Si se declaran sin signed, al compararlos con el int del modelo SystemVerilog los extiende con ceros a 32 bits y -14 (10 bits: 11_1111_0010)

se vuelve 1010; fallan todos los casos con resultado negativo y todos los de error (-1 se vuelve 1023) aunque el DUT esté bien. Se comprobó quitando el signed: aparecen FALLA con "esperado r=-14, obtenido r=-14", porque el print sí usa $signed pero la comparación no.

Testbench

- reloj de 5 MHz, la interfaz bus, el MDRS_wrapper conectado a bus, el handle mdrs_tester tester, y en el initial tester = new(bus), tester.reset() y los diez casos dirigidos, terminando con tester.summary() y $stop.
- Los diez casos cubren una operación normal y los errores de cada una:
- En el testbench no se toca ninguna señal. Como se manda: tester.div(-100, 7); todo lo demás lo hace la clase.
- El barrido exhaustivo no va aquí: Este tb es para mostrar el flujo interfaz → clase → modelo con casos que se puedan explicar uno por uno en la onda.